

Dennemeyer hirdetés kulcsszavak

Telefonról olvasható, magyarázatos jegyzet a Software Architect interjúhoz

Cél: ne csak felismerd a szavakat, hanem 1-2 percben értelmesen tudj róluk beszélni. A példák szándékosan API/Data Exchange Platform, payment, IP domain, Kubernetes és production reliability köré vannak építve.

Hogyan használd

- A vastagított fogalmakat tudd magyarul elmagyarázni, majd angolul 1 mondatban összefoglalni.
- A .NET résznél ne kamuzz mély hands-on tapasztalatot. A cél az, hogy ne legyen teljesen idegen a stack.
- A legerősebb saját kapcsolódásod: API contract, Node.js/TypeScript backend, WebSocket, Redis, Docker/Kubernetes, observability, rollout, mentoring.
- Interjún mindig trade-offban gondolkodj: miért ez a megoldás, mikor nem ezt választanád, milyen kockázat van.

Prioritás

Szint	Kulcsszavak
Must know	API platform, Data Exchange Platform, technical leadership, design document, microservices, DDD, CQRS, SOLID, Docker, Kubernetes, .NET stack, SQL, security standards
Jó ha tudod	OOA/OOD, TDD, Conway's Law, Scrum/Agile, NoSQL, IP domain, payment processing, product/customer goals
Saját erős pont	API contracts, idempotency, retries, rate limiting, observability, correlation IDs, health checks, graceful shutdown, rollout strategies, mentoring

1. Platform és API kulcsszavak

API (Application Programming Interface)

Mozaikszó: Application Programming Interface

Mit jelent? Egy szabályozott kommunikációs felület rendszerek között. Nem csak endpoint, hanem contract: milyen inputot vár, milyen outputot ad, milyen hibákat jelez, milyen auth kell.

Példa: Egy külső ügyfél hívja a POST /renewals endpointot, hogy elindítson egy trademark renewal folyamatot. Az API visszaad egy renewalId-t és egy PENDING státuszt.

Mire figyelj? Ne csak route-okról beszélj. Architect szinten az API stability, security, monitoring és customer integration is része.

Interjú fókusz: Mondd, hogy API-kat szerződésként kezelsz: validation, consistent error format, versioning, documentation, backward compatibility.

API and Data Exchange Platform

Mit jelent? A hirdetés központi eleme. Egy olyan platform, amely belső üzleti egységek és külső ügyfelek rendszerei között adatot cserél, validál, transzformál és továbbít.

Példa: Egy ügyfél saját rendszeréből elküldi a trademark adatait, a platform canonical belső formátumra alakítja, validálja, továbbítja egy internal service-nek, majd visszajelzi a státuszt webhookon vagy API-n.

Mire figyelj? A legnagyobb kockázat nem maga az endpoint, hanem a hibás adat, schema drift, duplikált request, jogosulatlan hozzáférés és rossz retry kezelés.

Interjú fókusz: A kulcsszavak: API contract, schema, data mapping, validation, idempotency, retry, audit log, authentication, authorization, rate limiting, observability.

Platform services

Mit jelent? Újrafelhasználható technikai vagy üzleti szolgáltatások, amelyeket több csapat vagy több termék is használ. Itt ilyen lehet API gateway, payment, customer integration, data exchange, audit vagy notification.

Példa: Nem minden business unit épít saját payment integrációt, hanem egy közös platform service kezeli a payment processinget.

Mire figyelj? A túl általános platform lassíthatja a product deliveryt. A túl specifikus platform nem lesz újrafelhasználható.

Interjú fókusz: Architectként azt kell mutatnod, hogy közös komponensekben gondolkodsz, de nem túlgeneralizálsz túl korán.

API contract

Mit jelent? Az API contract az API ígérete a kliensek felé: request mezők, response struktúra, hibakódok, auth szabályok, rate limit és verziózás.

Példa: Ha a response eddig {status: "ACTIVE"}, akkor nem cserélheted hirtelen {state: "active"}-re, mert eltörhetnek az ügyfél integrációk.

Mire figyelj? External customer API-nál a breaking change sokkal fájdalmasabb, mint belső API-nál.

Interjú fókusz: Hozd elő, hogy a CV-dben is szerepel API contracts, validation, consistent errors, pagination és contract testing.

OpenAPI / Swagger

Mit jelent? Az OpenAPI szabványos gépileg olvasható leírás REST API-khoz. A Swagger gyakran az OpenAPI dokumentáció megjelenítésére és tesztelésére használt tooling.

Példa: Leírod benne, hogy POST /renewals milyen body-t vár, milyen response-t ad, milyen 400/401/403/409/500 hibák lehetnek.

Mire figyelj? A dokumentáció akkor hasznos, ha követi a valós implementációt. Contract tesztekkel lehet védeni.

Interjú fókusz: External API platformnál ez fontos, mert az API dokumentáció maga is product feature.

Versioning és backward compatibility

Mit jelent? Versioning azt oldja meg, hogyan változtatod az API-t anélkül, hogy a régi kliensek eltörnének. Backward compatibility azt jelenti, hogy az új verzió még működik a régi kliensekkel.

Példa: Safe változás: új optional mező hozzáadása. Breaking változás: mező törlése, átnevezése, típusváltás, enum értékek váratlan módosítása.

Mire figyelj? Ne vezess ki endpointot hirtelen, ha ügyfél integráció épül rá.

Interjú fókusz: Mondd, hogy külső ügyfél API-nál deprecation policy, migration guide és átmeneti kompatibilitás kell.

Data mapping

Mit jelent? Data mapping során egyik adatstruktúrát alakítod másikra. API/Data Exchange Platformnál ez napi probléma.

Példa: Az egyik ügyfél `due_date` mezőt küld, a másik `renewalDueDate`-et. A belső modellben mindkettőből `dueDate` lesz.

Mire figyelj? Tipikus problémák: dátumformátum, currency, country code, optional mezők, enum eltérések.

Interjú fókusz: A mapping legyen explicit, tesztelt és verziózott, mert mapping hiba könnyen business hibává válik.

Canonical data model

Mit jelent? Egy közös belső adatmodell, amelyre a különböző külső formátumokat átalakítod. Így a belső service-eknek nem kell minden ügyfél egyedi formátumát ismerniük.

Példa: Customer A: `trade_mark_id`. Customer B: `trademarkId`. Belső canonical model: `TrademarkRenewal.trademarkId`.

Mire figyelj? Túl merev canonical model akadályozhat speciális ügyfél igényeket. Verziózás és extension mezők segíthetnek.

Interjú fókusz: Ez nagyon releváns data exchange platformnál. Jó architect téma.

Webhook

Mit jelent? Webhooknál nem a kliens kérdezi az állapotot, hanem a rendszer hívja vissza a kliens URL-jét, amikor esemény történt.

Példa: Renewal státusz COMPLETED lett, ezért a platform POST-olja az eventet a customer webhook endpointjára.

Mire figyelj? Webhooknál mindig számolj azzal, hogy a customer endpoint lassú vagy nem elérhető.

Interjú fókusz: Fontos szavak: signature validation, retry, timeout, idempotency, delivery log, event versioning.

Polling vs webhook

Mit jelent? Polling: a kliens időnként lekérdezi az állapotot. Webhook: a rendszer szól, ha változás van.

Példa: Polling: GET /renewals/123/status 30 másodpercenként. Webhook: a rendszer status changed eventet küld.

Mire figyelj? Webhook bonyolultabb, mert retry, security és delivery tracking is kell.

Interjú fókusz: Egyszerű integrációhoz polling elég lehet. Sok ügyfélnél és sok eseménynél webhook vagy event jobb.

2. Architecture és design kulcsszavak

Software architecture

Mit jelent? Architektúra alatt nem csak diagramot értünk, hanem döntéseket: komponensek, határok, adatfolyam, kommunikáció, failure handling, deployment, security és üzemeltethetőség.

Példa: Eldöntöd, hogy a renewal flow sync API hívás legyen-e, vagy async queue + status tracking. Ez architektúra döntés, nem csak implementáció.

Mire figyelj? Az architect nem mindenre a legkomplexebb megoldást választja. A jó döntés kontextusfüggő.

Interjú fókusz: Mutasd, hogy trade-offban gondolkodsz: egyszerűség, skálázhatóság, konzisztencia, költség, delivery speed.

Future-proof solutions

Mit jelent? Olyan megoldás, amely a várható növekedést és változást bírja, de nem túlépített. Future-proof nem azt jelenti, hogy mindenből enterprise monstnumot építünk.

Példa: Verziózott API, tiszta domain boundary, backward-compatible DB migration, monitoring, rollout/rollback terv.

Mire figyelj? Óvakodj az overengineeringtől. A túl általános design lassíthatja a csapatot.

Interjú fókusz: Mondd: future-proof = extensible, observable, secure, testable and deployable.

System design document

Mit jelent? Egy rövid, strukturált döntési dokumentum. Nem regény, hanem közös megértés: mit építünk, miért így, milyen alternatívák és riskek vannak.

Példa: Tartalom: goal, scope, functional requirements, non-functional requirements, diagram, API contract, data model, security, risks, rollout plan, monitoring.

Mire figyelj? A design doc ne legyen halott dokumentum. Review után frissíteni kell a döntések alapján.

Interjú fókusz: Ez szerepel a hirdetésben, ezért tudj róla beszélni. Mondd, hogy szereted a döntéseket dokumentálni, hogy később is érthető legyen a miért.

OOA (Object-Oriented Analysis)

Mozaikszó: Object-Oriented Analysis

Mit jelent? Objektumorientált elemzés. Itt még nem classokat kódolsz, hanem megérted a domain fogalmait, kapcsolatait és felelősségeit.

Példa: IP domainben azonosítod: Customer, Portfolio, Trademark, Patent, Renewal, Invoice, Payment, Document.

Mire figyelj? Ne keverd az OOA-t sima UML rajzolással. A cél a domain megértése.

Interjú fókusz: Mondd: először a business fogalmakat és szabályokat tisztázod, csak utána tervezel classokat vagy service-eket.

OOD (Object-Oriented Design)

Mozaikszó: Object-Oriented Design

Mit jelent? Objektumorientált tervezés. Az OOA-ban megértett fogalmakból classokat, interface-eket, modulokat és felelősségi határokat tervezel.

Példa: RenewalService nem közvetlenül SQL stringeket épít, hanem domain objektumokkal és repository interface-szel dolgozik.

Mire figyelj? A túl sok absztrakció is rossz. Csak ott érdemes absztrahálni, ahol tényleg változás várható.

Interjú fókusz: A lényeg: separation of concerns, low coupling, high cohesion, testability.

DDD (Domain-Driven Design)

Mozaikszó: Domain-Driven Design

Mit jelent? Domain-Driven Design. A rendszer szerkezete a business domain köré épül. A kód nyelve és a business nyelve közel legyen egymáshoz.

Példa: Trademark Renewal domainben a Renewal aggregate kezeli, hogy mikor indítható renewal, milyen státuszok vannak, milyen payment feltételek kellenek.

Mire figyelj? DDD-t nem kell minden CRUD appra ráerőltetni. Akkor értékes, ha a domain komplex.

Interjú fókusz: Tudd ezeket: bounded context, entity, value object, aggregate, repository, domain service, ubiquitous language.

Bounded context

Mit jelent? DDD fogalom. Egy határ, ahol egy business fogalom jelentése stabil. Ugyanaz a szó más contextben mást jelenthet.

Példa: Payment contextben a Customer lehet számlázási entitás. Portfolio contextben a Customer lehet IP portfólió tulajdonos.

Mire figyelj? Ha minden service ugyanazt az óriási shared modelt használja, könnyen tight coupling lesz.

Interjú fókusz: Microservice boundary tervezésnél nagyon hasznos fogalom.

CQRS (Command Query Responsibility Segregation)

Mozaikszó: Command Query Responsibility Segregation

Mit jelent? Command és Query felelősség szétválasztása. Command módosítja az állapotot, Query adatot olvas.

Példa: Command: SubmitRenewalRequest validációval és business rule-okkal. Query: RenewalDashboard gyors read modelből.

Mire figyelj? Ne mondd, hogy mindig használnád. Mondd: only when the complexity justifies it.

Interjú fókusz: Akkor hasznos, ha az írási és olvasási igények nagyon mások, például dashboard sok szűréssel, komplex írási workflow mellett.

SOLID

Mit jelent? Öt objektumorientált design alapelv: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. Cél: érthetőbb, tesztelhetőbb, lazábban csatolt kód.

Példa: Single Responsibility: a PaymentFeeCalculator ne küldjön emailt és ne írjon audit logot is. Mindegyik külön felelősség.

Mire figyelj? SOLID túlzásba véve absztrakciós káoszt okozhat. Pragmatikusan kezeld.

Interjú fókusz: Interjún nem kell tankönyvszerűen mind az ötöt mélyen. Magyarázd úgy: kisebb felelősségek, dependency injection, interface-ek, könnyebb tesztelés.

Conway's Law

Mit jelent? Conway törvénye szerint a rendszer architektúrája gyakran leképezi a szervezet kommunikációs struktúráját.

Példa: Ha külön team felel paymentért, customer integrationért és IP portfolioért, akkor ezek körül természetesen service boundary-k alakulhatnak ki.

Mire figyelj? Ha rossz csapathatárookra építész service-eket, sok cross-team dependency és lassú delivery lehet.

Interjú fókusz: Architectként ne csak technikai, hanem ownership és kommunikációs határok alapján is tervezz.

3. .NET kulcsszavak, a nagy gap kezelése

A hirdetés erős .NET tudást kér. Neked nem kell úgy tenned, mintha ez lenne a fő stacked. A cél: ismerd a fogalmakat, és tudd Node.js/NestJS/Express párhuzammal elhelyezni őket.

.NET technology stack

Mit jelent? A Microsoft backend ökoszisztémája: runtime, framework, tooling, package management, web framework, ORM, testing és deployment lehetőségek.

Példa: Egy enterprise REST API épülhet C# + ASP.NET Core + Entity Framework + SQL Server/PostgreSQL stacken.

Mire figyelj? Ne állítsd, hogy excellent .NET hands-on tudásod van. Inkább ramp-up time-ról beszélj.

Interjú fókusz: Mondd: a recent hands-on stacked Node.js/TypeScript, de az API, architecture, DI, ORM, middleware és deployment minták ismerősek.

C#

Mit jelent? A .NET világ fő programozási nyelve. Erősen típusos, objektumorientált, modern async/await és generics támogatással.

Példa: Nálad a TypeScript tölti be a napi fő nyelv szerepét. C# a .NET backend világ TypeScript/Java párhuzama.

Mire figyelj? A syntaxot nem kell mélyen tudni egy architect callhoz, de az ökoszisztémában elfoglalt szerepét igen.

Interjú fókusz: Tudd kimondani: C# is the main language used for ASP.NET Core backend services.

ASP.NET Core

Mit jelent? Modern .NET web framework REST API-khoz, web appokhoz és microservice-ekhez.

Példa: Node-os párhuzam: Express vagy NestJS szerepe, csak erősen enterprise standard Microsoft környezetben.

Mire figyelj? A régi ASP.NET/.NET Framework és modern ASP.NET Core nem ugyanaz. Modern projektnél Core/.NET a releváns.

Interjú fókusz: Tudd: controller, routing, middleware, dependency injection, auth, configuration, logging.

Web API

Mit jelent? .NET kontextusban általában REST API ASP.NET Core alatt.

Példa: Controller fogadja: POST /renewals, meghívja a service layer-t, visszaadja a DTO response-t.

Mire figyelj? A controller ne tartalmazzon túl sok business logicot. Az menjen service/domain layerbe.

Interjú fókusz: Node párhuzam: Express route handler vagy NestJS controller.

Entity Framework Core (EF Core)

Mozaikszó: EF Core

Mit jelent? A .NET legismertebb ORM-je. ORM = Object Relational Mapper. SQL táblákat C# objektumokon keresztül kezelsz.

Példa: `db.Renewals.Where(r => r.CustomerId == id)` mögött SQL query fut.

Mire figyelj? ORM nem varázslat. Rossz queryt generálhat, N+1 problémát okozhat, indexekre ugyanúgy figyelni kell.

Interjú fókusz: Node párhuzam: TypeORM, Prisma vagy Sequelize. Architect szinten a performance, lazy loading, query plan és migration fontos.

LINQ (Language Integrated Query)

Mozaikszó: Language Integrated Query

Mit jelent? C# query nyelvi funkció gyűjteményekhez és EF Core esetén adatbázis-lekérdezésekhez.

Példa: `users.Where(u => u.IsActive).OrderBy(u => u.Name)` lehet memória művelet vagy EF esetén SQL-re forduló query.

Mire figyelj? EF-nél nem minden LINQ expression fordítható hatékony SQL-re.

Interjú fókusz: Tudd: LINQ is the standard C# way to express queries over collections and database-backed entities.

Dependency Injection (DI)

Mozaikszó: DI

Mit jelent? A class nem maga hozza létre a dependency-jeit, hanem kívülről kapja meg. Ez lazább couplingot és könnyebb tesztelést ad.

Példa: `RenewalService` constructorban kap `IPaymentService`-t és `IRenewalRepository`-t. Tesztben ezek mockolhatók.

Mire figyelj? DI nem cél önmagában. Ha túl sok interface van valós csereigény nélkül, zajt okozhat.

Interjú fókusz: ASP.NET Core-ban beépített alapl minta. NestJS-ben hasonlóan természetes.

Middleware

Mit jelent? Request feldolgozási lánc egyik lépése. Minden request átmehet logging, auth, CORS, error handling, tracing middleware-en.

Példa: Request -> logging -> auth -> validation -> controller -> response.

Mire figyelj? A business logic ne middleware-be kerüljön, kivéve ha valóban cross-cutting concern.

Interjú fókusz: Expressből ismerős. ASP.NET Core-ban is hasonló pipeline elv van.

NuGet

Mit jelent? .NET package manager. Node.js világban az npm megfelelője.

Példa: Logging library, DB driver, testing library vagy cloud SDK NuGet package-ként jöhet.

Mire figyelj? Enterprise környezetben package audit és dependency update policy is fontos.

Interjú fókusz: Elég tudni, hogy dependency/package management eszköz a .NET ökoszisztémában.

Kestrel

Mit jelent? ASP.NET Core beépített web szervere. Ez fogadja a HTTP requesteket az appon belül.

Példa: Client -> Load Balancer / Reverse Proxy -> Kestrel -> ASP.NET Core app.

Mire figyelj? Productionben gyakran reverse proxy vagy load balancer van előtte.

Interjú fókusz: Elég annyi: Kestrel is the built-in web server used by ASP.NET Core apps.

4. Container, deployment és operations kulcsszavak

Container technologies

Mit jelent? Container technológiák célja, hogy az app ugyanazzal a csomagolással fusson dev, staging és production környezetben. Ide tartozik Docker, Kubernetes, registry, image scanning, secrets, health checks.

Példa: Egy API service Docker image-ként épül Jenkinsben, registrybe kerül, majd Kubernetes deployment frissíti rolling update-tel.

Mire figyelj? A container nem old meg rossz app design-t. Stateless, config, shutdown, logs továbbra is fontos.

Interjú fókusz: Hirdetésben szerepel, ezért készülj: Docker image, Kubernetes pod, deployment, service, ingress, readiness/liveness, rollback.

Docker

Mit jelent? Containerizálás eszköze. Docker image tartalmazza az appot, runtime-ot, dependency-ket és indulási parancsot.

Példa: Node app-nál Dockerfile telepíti a dependency-ket, buildel, majd production image indul npm starttal.

Mire figyelj? Figyelni kell image méretre, security scanre, secret ne kerüljön image-be.

Interjú fókusz: Mondd: Docker segít környezeti különbségek csökkentésében és CI/CD pipeline-ban.

Kubernetes (K8s)

Mozaikszó: K8s

Mit jelent? Container orchestration platform. Podokat futtat, skáláz, újraindít, service discoveryt és rolloutot kezel.

Példa: Ha egy pod meghal, Kubernetes új podot indíthat. Rolling update-nél fokozatosan cseréli a régi verziót az újra.

Mire figyelj? A Kubernetes komplex. Csak akkor érték, ha a team tudja üzemeltetni.

Interjú fókusz: Tudd: pod, deployment, service, ingress, configmap, secret, readiness probe, liveness probe, horizontal pod autoscaler.

Readiness és liveness probe

Mit jelent? Readiness: kaphat-e forgalmat az app. Liveness: életben van-e még, vagy újra kell indítani.

Példa: Deploy után a pod csak akkor kap requestet, ha DB connection és config rendben van. Ha deadlock miatt nem válaszol, liveness restartolhatja.

Mire figyelj? Ha rosszul állítod be, Kubernetes restart loopot vagy hibás traffic routingot okozhat.

Interjú fókusz: Zero downtime rolloutnál readiness + graceful shutdown kulcsfontosságú.

Rolling update, canary, blue-green

Mit jelent? Deployment stratégiák. Rolling update fokozatosan cserél. Canary kis forgalmon tesztel. Blue-green két teljes környezet között vált.

Példa: Új renewal API verzió először 5% trafficot kap canaryként, metrics alapján döntünk a teljes rolloutól.

Mire figyelj? Feature flag nélkül a rollback néha nem elég, főleg ha DB migration is történt.

Interjú fókusz: A CV-ben ez erős pont: blue-green, canary, shadow traffic, Jenkins, Kubernetes rollout.

CI/CD (Continuous Integration / Continuous Delivery)

Mozaikszó: Continuous Integration / Continuous Delivery

Mit jelent? CI: gyakori integráció automatikus builddel és tesztekkel. CD: kontrollált, automatizált delivery/deployment staging vagy production felé.

Példa: Push után Jenkins futtat testeket, buildel Docker image-et, majd staging deploy következik.

Mire figyelj? A gyors pipeline semmit sem ér, ha nincs minőségi teszt és release kontroll.

Interjú fókusz: Interjún hangsúlyozd: safe release, repeatable build, test gates, rollback, visibility.

Secure deployment

Mit jelent? Deployment úgy, hogy ne sérüljön security: secret kezelés, least privilege, image scanning, TLS, network policy, audit, rollback.

Példa: API service csak a saját DB credentialjét kapja, nem teljes admin jogosultságot. Secret nem Gitben van, hanem secret managerben.

Mire figyelj? A production config ne dev default legyen. Debug endpointok és verbose errorok veszélyesek.

Interjú fókusz: Kapcsold össze security standards és infrastructure teams részekkel a hirdetésből.

Maintain applications

Mit jelent? Nem csak lefejleszteni kell, hanem üzemeltethetővé tenni: monitoring, alerting, logs, dependency update, capacity planning, incident response.

Példa: Egy API service-hez dashboard, alert, runbook és rollback lépés is tartozik.

Mire figyelj? A maintainability architect szinten tervezési szempont, nem utólagos extra.

Interjú fókusz: Ez nálad erős: Prometheus, Grafana, Sentry, Kibana, structured logging, health checks, production support.

5. Data, SQL és NoSQL kulcsszavak

SQL (Structured Query Language)

Mozaikszó: Structured Query Language

Mit jelent? Relációs adatbázisok lekérdezési nyelve. Táblák, sorok, oszlopok, joinok, tranzakciók és indexek világa.

Példa: Renewals tábla joinolható Customers és Payments táblával, hogy megkapjuk egy ügyfél aktív megújításait.

Mire figyelj? Ne csak azt mondd, hogy használtál SQL-t. Architect szinten performance és adatkonzisztencia is fontos.

Interjú fókusz: Tudd: index, transaction, isolation, join, query plan, migration, normalization, constraints.

Index

Mit jelent? Adatbázis gyorsított keresési struktúra. Olyan, mint egy könyv tárgymutatója. Gyorsítja az olvasást, de lassíthatja az írást.

Példa: Ha gyakran keresel customerId + status + dueDate alapján, composite index segíthet.

Mire figyelj? Túl sok index növeli storage-t és lassítja insert/update műveleteket.

Interjú fókusz: Interjún: indexet query pattern alapján választunk, nem érzésre.

Transaction és ACID

Mit jelent? Transaction több DB műveletet egy logikai egységként kezel. ACID: Atomicity, Consistency, Isolation, Durability.

Példa: Payment státusz frissítése és renewal request mentése együtt sikerül vagy együtt rollbackel.

Mire figyelj? Microservice-ek között distributed transaction nehéz. Ott saga/outbox/eventual consistency jöhet.

Interjú fókusz: Tudd: transaction akkor kell, ha félkész állapot business hibát okozna.

Isolation level

Mit jelent? Azt szabályozza, hogy párhuzamos transactionök mennyire láthatják egymás félkész vagy változó adatait.

Példa: Magasabb isolation csökkenti furcsa olvasásokat, de több lockot és lassulást okozhat.

Mire figyelj? Nem kell mindenhol serializable. Business kritikalitás alapján kell dönteni.

Interjú fókusz: Interjúban elég: consistency vs performance trade-off.

Database migration

Mit jelent? DB schema kontrollált változtatása. Productionben backward-compatible lépések kellene.

Példa: Új oszlop hozzáadása, app dual-write, backfill, read switch, régi oszlop törlése később.

Mire figyelj? Egy rossz migration teljes production downtime-t okozhat.

Interjú fókusz: API és DB változásokat release tervvel kezeld.

NoSQL

Mit jelent? Nem relációs adatbázis kategória. Ide tartozhat document store, key-value store, wide-column, graph DB. A hirdetés assetként említi.

Példa: MongoDB document alapú ügyfél konfigurációhoz, Redis cache/pubsub-hoz, DynamoDB key-based lookuphoz.

Mire figyelj? NoSQL nem automatikusan gyorsabb. Rossz access patternnel ugyanúgy fájdalmas.

Interjú fókusz: Tudd SQL vs NoSQL trade-offot: schema, transactions, joins, scaling, access pattern.

Redis

Mit jelent? In-memory data store, gyakran cache, pub/sub, rate limiting, session vagy distributed lock célra.

Példa: Nálad erős példa: WebSocket chat Redis pub/sub-bal horizontálisan skálázva.

Mire figyelj? Redis memóriaalapú, ezért eviction, persistence és failure mode fontos.

Interjú fókusz: Kapcsold cachinghez, real-time messaginghez, rate limithez és distributed state-hez.

Data boundaries

Mit jelent? Service-ek vagy bounded context-ek közötti adat tulajdonjogi határ. Ki írhatja, ki olvashatja, ki az adat source of truth?

Példa: Payment status source of truth a payment service. Más service csak eventből vagy API-ból kapja meg.

Mire figyelj? Shared database több service között gyorsnak tűnik, de hosszú távon tight couplingot okozhat.

Interjú fókusz: Microservices és DDD mellett fontos. Ne legyen minden service ugyanarra a táblára írva.

6. Security és quality kulcsszavak

Quality standards

Mit jelent? A minőség nem csak teszt. Ide tartozik code review, automated tests, observability, security checks, API contract stability, performance, maintainability.

Példa: Egy renewal API feature akkor kész, ha van validáció, error handling, teszt, log, monitoring és dokumentált contract.

Mire figyelj? A gyors delivery minőség nélkül később production incidentet okoz.

Interjú fókusz: A hirdetés quality goalokat említ, ezért mutasd, hogy product-ready rendszerekben gondolkodsz.

Security standards

Mit jelent? Következetes szabályok érzékeny rendszerek védelmére: auth, authorization, encryption, audit logging, secret management, least privilege, input validation.

Példa: Customer API key csak adott customer portfólióját érheti el. Minden módosítás audit logba kerül.

Mire figyelj? Security ne legyen csak utolsó penetration test. Már designkor szerepeljen.

Interjú fókusz: API/Data Exchange Platformnál ez kritikus, mert külső integrációk és üzleti/jogi adatok vannak.

Authentication és Authorization

Mit jelent? Authentication = ki vagy? Authorization = mit tehetsz meg?

Példa: A token alapján tudjuk, hogy melyik customer app hív. Authorization alapján döntjük el, elérheti-e a megadott portfóliót.

Mire figyelj? Sikeres login nem jelent automatikusan minden adathoz hozzáférést.

Interjú fókusz: Tudd elkülöníteni a kettőt. Ez gyakori interjúkérdés.

OAuth, OIDC, JWT

Mit jelent? OAuth: authorization framework. OIDC, OpenID Connect: authentication layer OAuth fölött. JWT, JSON Web Token: token formátum claim-ekkel és aláírással.

Példa: OIDC megmondja, ki a user vagy client. OAuth scope megmondja, mit tehet. JWT hordozhatja a claim-eket.

Mire figyelj? JWT-ben ne tárolj túl sok érzékeny adatot. Expiry, revoke, key rotation fontos.

Interjú fókusz: A CV-dben JWT és OAuth/OIDC szerepel, ezt használd hiteles kapcsolódásként.

RBAC (Role-Based Access Control)

Mozaikszó: Role-Based Access Control

Mit jelent? Jogosultság szerepkör alapján. Például Admin, Support Agent, Customer, Read-only User.

Példa: Support Agent láthat ügyfél ticketeket, de nem módosíthat payment adatot.

Mire figyelj? Sok kivételnél túl merev lehet. Ilyenkor ABAC segíthet.

Interjú fókusz: Egyszerű, jól érthető modell. Jó belső admin rendszerekhez.

ABAC (Attribute-Based Access Control)

Mozaikszó: Attribute-Based Access Control

Mit jelent? Jogosultság attribútumok alapján. Nem csak role, hanem user, customer, resource, country, sensitivity, ownership alapján dönt.

Példa: User akkor láthat documentet, ha user.companyId == document.companyId és document.sensitivity nem restricted.

Mire figyelj? Bonyolultabb tesztelni és debugolni. Kell policy logging.

Interjú fókusz: Rugalmasabb, mint RBAC, customer/tenant alapú rendszerekben hasznos.

Least privilege

Mit jelent? Minden user, service és job csak a minimális jogosultságot kapja, ami tényleg kell.

Példa: Reporting service csak olvasási DB jogot kap, nem törölhet vagy módosíthat production adatot.

Mire figyelj? Admin tokenek és túl széles jogosultságok incidentnél nagy kárt okoznak.

Interjú fókusz: Service-to-service integrációnál scoped tokenek és külön service accountok kellenek.

Input validation

Mit jelent? Minden bejövő adat ellenőrzése backend oldalon. Required mezők, típusok, enumok, hossz, dátum, jogosultság.

Példa: renewalDate nem lehet invalid dátum, customerId-hez a hívónak jogosultnak kell lennie.

Mire figyelj? Validáció és business rule nem ugyanaz. Egy dátum formailag valid lehet, de üzletileg már túl késő renewalhoz.

Interjú fókusz: Frontend validáció nem elég. API-t közvetlenül is lehet hívni.

Audit logging

Mit jelent? Üzleti vagy security szempontból fontos események bizonyítható naplózása. Nem ugyanaz, mint debug log.

Példa: Ki módosította a trademark státuszt, mikor, milyen előző és új értékkel, milyen requestId-val.

Mire figyelj? Audit log legyen kereshető és lehetőleg ne legyen könnyen módosítható. Ne tartalmazzon secretet.

Interjú fókusz: IP/payment/customer adatoknál nagyon releváns.

TDD és testing strategy (Test-Driven Development)

Mozaikszó: Test-Driven Development

Mit jelent? TDD szigorúan azt jelenti: előbb teszt, aztán implementáció. Interjún praktikusabb testing strategyként beszélni róla.

Példa: Unit test business rule-ra, integration test DB/API-ra, contract test API schema-ra, E2E test kritikus user flow-ra.

Mire figyelj? Nem minden teszt egyformán értékes. Kritikus domain logic és contract stability fontosabb, mint 100% coverage mutogatása.

Interjú fókusz: Nálad jó kapcsolódás: Jest, React Testing Library, Selenium, contract testing, Supertest.

7. Process, product és leadership kulcsszavak

Technical leadership

Mit jelent? Technikai irányadás title-től függetlenül: requirement tisztázás, trade-offok, design döntések, code review, mentoring, unblockolás, kommunikáció.

Példa: Egy komplex API feature-t kisebb deliverable-ökre bontasz: contract, DB migration, service logic, tests, monitoring, rollout.

Mire figyelj? Ne csak people managementként értelmezd. Ez technical decision making és team enablement.

Interjú fókusz: Ez a legfontosabb pozicionálási pontod. Mondd: formal architect title nélkül is csináltál architecture-level feladatokat.

Coach and lead development team

Mit jelent? A hirdetésben ez azt jelenti, hogy segítened kell a fejlesztőknek jó döntéseket hozni, reviewzni, standardokat tartani, és a technikai irányt érthetővé tenni.

Példa: Junior/mid developer bizonytalan API error handlingben, te adsz mintát, reviewzod, dokumentálsz a standardot.

Mire figyelj? Coaching nem mikromenedzsment. A cél, hogy a csapat önállóbb legyen.

Interjú fókusz: Használd a CV-d mentoring, onboarding, pair programming, code review elemeit.

Work breakdown és deliverables

Mit jelent? Nagy feladat lebontása kisebb, ellenőrizhető részekre. Deliverable = kézzelfogható, tesztelhető eredmény.

Példa: API platform feature: 1. OpenAPI contract, 2. DB migration, 3. mapping logic, 4. auth, 5. tests, 6. monitoring, 7. rollout.

Mire figyelj? Túl nagy ticketek lassítják a reviewt és növelik a risket.

Interjú fókusz: Architect/tech lead szerepben ez kulcs. Nem elég megmondani az architektúrát, végrehajtható terv kell.

Scrum / Agile

Mit jelent? Iteratív fejlesztési mód. Sprint, backlog, refinement, planning, daily, review, retrospective.

Példa: Product managerrel refinementen tisztázod a business igényt, majd technikai taskokra bontod.

Mire figyelj? Agile nem jelenti, hogy nincs design. Csak a design legyen elég részletes a következő lépésekhez.

Interjú fókusz: Mondd: agile nálad nem meetinglista, hanem rövid feedback loop és priorizált delivery.

Product managers és business line lead

Mit jelent? A hirdetés szerint product/business oldallal szorosan dolgoznál. Architectként üzleti célokat kell technikai döntésekre fordítanod.

Példa: Business cél: külső ügyfelek könnyebben integrálják a renewal API-t. Tech feladat: stabil contract, dokumentáció, sandbox, webhook, monitoring.

Mire figyelj? A túl technikai magyarázat product oldalnak elveszhet. Business impact nyelvre is le kell fordítani.

Interjú fókusz: Mutasd, hogy tudsz nem-technikai szereplőkkel kommunikálni.

Revenue, customer satisfaction, quality goals

Mit jelent? A hirdetés nem csak technikai outputot vár, hanem product eredményt: bevétel, elégedett ügyfél, megbízható minőség.

Példa: Ha az API stabil és jól dokumentált, az ügyfél gyorsabban integrál, kevesebb support kell, jobb customer satisfaction.

Mire figyelj? A tökéletes technikai design sem jó, ha lassítja a piacra kerülést vagy nem oldja meg az ügyfél problémáját.

Interjú fókusz: Interjún érdemes mondani: architecture decisions should support product outcomes.

Entrepreneurial mindset

Mit jelent? Önálló, felelősségvállaló gondolkodás. Nem csak ticketeket hajtasz végre, hanem keresed az üzleti és technikai hatást.

Példa: Észreveszed, hogy a customer onboarding lassú az API dokumentáció hiánya miatt, és javasolsz sandbox + OpenAPI javítást.

Mire figyelj? Ne hangozzon úgy, mintha mindent egyedül akarnál eldönteni. Entrepreneurial + collaborative együtt jó.

Interjú fókusz: Mondd: szeretted azokat a feladatokat, ahol technikai döntésnek mérhető business hatása van.

Customer-focused mindset

Mit jelent? Nem csak belső kódminőség, hanem ügyfélélmény. API platformnál az ügyfél fejlesztője is user.

Példa: Jó error message, jó dokumentáció, sandbox environment, predictable API behavior, clear rate limits.

Mire figyelj? A customer-focused nem azt jelenti, hogy minden egyedi kérést rögtön custom hackkel oldasz meg. Platform szemlélet kell.

Interjú fókusz: Kiemelten releváns Dennemeyer külső customer integration céljához.

Cross-team communication

Mit jelent? Több csapat, product, infra, QA, business együttműködése. Architectként sok függőséget kell tisztázni.

Példa: API feature érint frontend teamet, backend service-t, infra deployt, security reviewt és customer support dokumentációt.

Mire figyelj? A rossz kommunikáció architecture problémává válik: rework, duplikált megoldások, breaking change.

Interjú fókusz: Hozd a CV-ből: QA-val rollout koordináció, backend/front-end együttműködés, dokumentáció, onboarding.

8. IP domain és business kulcsszavak

IP (Intellectual Property)

Mozaikszó: Intellectual Property

Mit jelent? Szellemi tulajdon. Ide tartozik például patent, trademark, design és copyright. Dennemeyer ebben az iparágban dolgozik.

Példa: Egy cég védi a márkanévét trademarkkal, egy találmányát patenettel, egy termék kinézetét design joggal.

Mire figyelj? Ne keverd az IP címmel. Itt nem Internet Protocolról van szó.

Interjú fókusz: Nem kell jogásznak lenned, de értsd, hogy ez adatintenzív, határidős, compliance és dokumentum-orientált domain.

Patent

Mit jelent? Szabadalom. Találmány vagy technikai megoldás jogi védelme adott országokban/régiókban.

Példa: Egy gyógyszeripari vagy tech cég patent portfóliót kezel, határidőkkel, díjakkal, státuszokkal.

Mire figyelj? Jogászcodni nem kell, csak domain complexityként kezeld.

Interjú fókusz: System design szempontból: komplex lifecycle, határidők, dokumentumok, országonként eltérő szabályok.

Trademark

Mit jelent? Védjegy. Márkanév, logó vagy jelölés védelme.

Példa: Egy cég bejegyezteti a logóját és időnként meg kell újítania a védjegyet.

Mire figyelj? A státuszok és határidők pontos kezelése üzletileg kritikus.

Interjú fókusz: API példákban jó: trademark renewal, trademark status, trademark portfolio.

Renewal

Mit jelent? Megújítás. IP jogoknál sok dolognak lejáratí és megújítási határideje van.

Példa: Trademark renewal elindul, payment megtörténik, dokumentum generálódik, státusz változik PENDING-ről COMPLETED-re.

Mire figyelj? Hibás renewal kezelés jogi/business kárt okozhat.

Interjú fókusz: Ez tökéletes system design példa: workflow, payment, status, audit, notification, retry, eventual consistency.

Portfolio management

Mit jelent? Egy ügyfél összes IP jogának, státuszának, határidejének és dokumentumának kezelése.

Példa: Egy multinacionális ügyfélnek több ezer trademarkja és patentje lehet sok országban.

Mire figyelj? Nagy portfóliónál list endpointok, indexek és filtering nagyon fontosak.

Interjú fókusz: Data model, search, pagination, authorization, audit és reporting témákhoz kapcsolódik.

Backbone payment processing technology

Mit jelent? A hirdetés említi a Group backbone payment processing technology-t. Ez valószínűleg központi payment szolgáltatás más Denemeyer egységek és külső integrációk felé.

Példa: Renewal díj fizetése után payment status event frissíti a renewal workflow-t.

Mire figyelj? Paymentnél nem szabad fallbackként sikeres státuszt hazudni. Pending/retrying/failed állapot kell.

Interjú fókusz: Payment témáknál hangsúlyozd: idempotency, audit log, strong consistency ott, ahol pénzügyi állapot kritikus, retry óvatosan.

9. Rövid angol mondatok interjúra

My recent hands-on stack is mainly Node.js and TypeScript, but many of my responsibilities were architecture-level: API contracts, service design, data boundaries, failure handling, observability, Docker/Kubernetes deployments and mentoring.

For an external API platform I would treat the API as a product: stable contracts, clear documentation, authentication, rate limiting, audit logs and predictable error handling.

Before choosing an architecture, I would clarify the non-functional requirements: traffic, latency, security, auditability, availability and integration constraints.

I would use DDD and CQRS pragmatically, where the domain complexity or read/write patterns justify it, not as default complexity.

For payment or renewal workflows I would pay special attention to idempotency, retries with backoff, audit logs and explicit states such as pending or retrying.

I would need ramp-up time on .NET implementation details, but the backend architecture patterns around APIs, DI, middleware, ORM, testing and deployment are familiar to me.

As a technical leader, I try to make decisions actionable for the team: clear contracts, small deliverables, documented trade-offs and early feedback loops.

10. Ha csak 30 perced van

- Tanuld meg a .NET gap kezelő mondatot.
- Ismételd át: API contract, Data Exchange Platform, DDD, CQRS, SOLID, Docker/K8s, SQL, security standards.
- Készíts 2 saját sztorit: high-traffic WebSocket/Redis rendszer, és API contract/reliability/observability történet.
- A recruiter/manager számára ne túl mély technikai részleteket mondj, hanem business hatást is: customer integration, quality, reliability, delivery.

Forrás

A jegyzet a feltöltött Dennemeyer Software Architect hirdetés kulcsszavaiból és a feltöltött Richard Sallai Full Stack CV kapcsolódó tapasztalataiból készült.